

Functions

1. What are functions?
2. Creating your own functions
3. Namespaces
4. Multiple Returns
5. Default Parameters
6. Keyword arguments
7. *args
8. Lambda
9. Additional Keywords in Python
10. Definitions

What are functions?

- Programmers hate to repeat things since it spams their source code with lines of code that could probably be easily avoided
- Therefore each repeating process that is NOT part of a loop should be written as a function
- Saves multiples in the source code for repeating processes during the program and allows you to use the same approach later on in other scripts as well by simply copying your function to your new script
- Doing so results in a better structure and outline of the code and thus improves readability greatly

The signature of a function describes only, which name the function has and which parameters it uses. The programming of a function, that means the source code of it, isn't displayed in a signature.

```
def sum_up(num1, num2):  
    result = num1 + num2  
    return result
```

- Many functions need some variables to fulfill their task
- These variables are called parameters and their names are defined in the signature of a function
- All functions should only use these variables for their calculations and therefore should only depend on them
- Using a functions with values of the wrong type as parameters will lead to either unwanted results or error messages, so make sure to name your parameters in a way that describes their datatype or datastructure type

- The inside of a function can be seen as a completely separate script that is only executed when we need it to be executed
- Therefore we can use all programming constructs we learned so far inside our functions as well, to create rather complex algorithms in there
- We can execute other functions inside our defined function as well
- This is not limited to built-in functions but can also be done with functions we defined on our own
- All variables we declare and assign in our functions are only available to us inside these functions, we can NOT access their values outside of our functions, that means in the rest of our script

- After we defined a function and created the algorithm inside it we can later execute it in our script
- This is done by a process named calling a function / call of a function and you already did this multiple times during this course
- Everytime you use the print command for example you called the function print with any number of arguments
- So when we define our own functions we can call them in the same manner, of course we need to make sure that the arguments fit the needs of our function
- The values of our arguments are then assigned to the parameters and our function is executed while using these values

- In most cases we create functions that run specific calculations for us and therefore produce at least one specific result
- As mentioned we are not able to access the value of these results outside of our function, that means we need to find a way to make these results accessible
- This can be done by handing over the value of our result to our main script with the help of a so called return statement
- We already observed this process when we used built-in functions like `len()` which we could use to determine the number of elements in a datastructure
 - Here we got back a single integer value which we could assign to a new variable for example

Creating your own functions

- In Python functions are defined with the keyword `def`
- The statement `def` is followed by a space and the name of the function with the parameters passed to the function in brackets
- This line is the signature of our function and needs to be closed with a colon at the end, similar to loops or if statements
- Let's say we want to create a function named `sum_up` that uses two parameters `num1` and `num2`, then we would need to write the following signature:

```
def sum_up(num1, num2):
```

- Now we can add an algorithm to our function by writing it inside of our function
- These algorithms can contain everything we used so far and follow the same rules we learned beforehand
- Important is that everything we want to be part of our function is indented so that python knows that it is inside of it
- Again all variables we define here are only available while the function is executed and not outside of it:

```
def sum_up(num1, num2):  
    result = num1 + num2
```

- The execution of a function is stopped as soon as the indented statements end or the keyword `return` is encountered
- With the help of the keyword `return` we are able to hand back the result of our function to the main script
- Only when using `return` we are able to use the results of our functions later on in our main script:

```
def sum_up(num1,num2):  
    result = num1 + num2  
    return result
```

- After we created our function, we want to execute it and we can do so by calling it
- Here we use the name of our function and write the values we want to assign to the parameters in brackets afterwards as arguments of our function
- If our function returns a value to the main script we can declare a variable to be assigned the result of our function as it's shown below:

```
sum1 = sum_up(5, 6)
```

- In this example the parameter num1 is assigned the value 5 and num2 is assigned the value 6
- The returned value of result is then assigned to our new variable sum1

```
def sum_up(num1,num2):  
    result = num1 + num2  
    return result
```

➔ num1 is set to 5 and num2 is set to 6

➔ result = 5 + 6

➔ return 11

```
sum1 = sum_up(5,6)
```

➔ execute sum_up with the numbers 5 and 6

```
sum2 = sum_up(20,100)
```

➔ execute sum_up with the numbers 20 and 100

```
sum3 = sum_up(5.2,7.7)
```

➔ execute sum_up with the numbers 5.2 and 7.7

```
print("I'm done")
```

- When using functions you often have variables in these repeated code parts
- Programs deal with them in a special way
- So far we encountered global variables, that means after we declared them they were available for usage throughout our remaining script
- Functions on the other hand use a special type called local variables
 - ▶ These only exist when the function runs
 - ▶ When a local variable has the same name as another variable (e.g. a global variable), the local one hides the other
 - ▶ That means we use the value we assigned to it inside our function while the function is executed but the other value everywhere else

- Create functions to avoid duplicate source code, that means as soon as you have to calculate the same thing twice you should define a new function
- A function should only depend on her parameters and therefore use only these to finish its calculations
 - If you find that your function needs access to global variables it's better to change your signature
- One function is exactly one job, that does NOT mean however that it produces only one result
- The source code shouldn't contain too much lines, so try to split up functions where it's possible

- Please stick to the naming conventions we learned when we were talking about Variables and use descriptive function names
- Please refrain from using names like `sum` or `reversed` which are identical to built-in Python functions
- Otherwise it could happen that you overwrite these functions with your own ones, the same is true for variable names so refrain from using names like `sum` here as well
- Use something like `addition` or `reverse_list` instead

- Create a program that defines functions for the three remaining mathematical basic operations ($-$, $*$ and $/$)
- Call each of the functions at least three times with different arguments

- These simple functions we generated previously took primitives (numbers or strings) as parameters and then calculated something
- We can also hand over datastructures as parameters to our functions and then run for example a loop over them if needed
- We can use any type of variable or datastructure for our parameters

- Write a program that defines functions for the following tasks:
 - ▶ Multiply all elements with each other in a list of numbers (element1 * element2 * element3 * ... * element N)
 - ▶ Add up all numbers inside a range defined by a start and end value (range function and two parameters)
 - ▶ Create a list containing the square numbers of the elements of a list of numbers. (Input: [1,2,3] → Output: [1,4,9])
- Each function should use a for loop to fulfill its task
- Call each function at least twice

- Write a program that defines functions for the following tasks:
 - ▶ Multiply each element of a list of numbers with each element of a second list of numbers
($\text{num1} \cdot \text{num1}, \text{num1} \cdot \text{num2}, \dots, \text{num2} \cdot \text{num1}, \text{num2} \cdot \text{num2}, \dots, \text{numN} \cdot \text{numN}$)
 - ▶ Print out the multiplication of two numbers from different ranges starting at 1 and ending at a number specified by the parameters (`range(1,num1)` & `range(1,num2)`)
- Each function should use nested for loops to fulfill its task
- Call each function at least once

Namespaces

- When talking about functions we have to talk about namespaces
- I mentioned this term as well as the terms global and local variables earlier on
- We also heard about namespaces when we were looking at the Zen of Python:
 - Namespaces are one honking great idea — let's do more of those!
- But what are they?

- First we have to make clear what we mean with Names
- A name or identifier is given to objects when we define / declare things like variables, data structures or functions in python
- In python, we can declare variable names and assign them to the objects as we saw multiple times already either with primitives like numbers or more complex things like data structures
- For functions we are doing the same, since our signature creates a new function with the name we specified in there

- A namespace is a collection of names and the details of the objects referenced by these names
 - ▶ That means it includes all possible methods we can use on our data structure, for example list methods like `append()` or `extend()` can also be part of a namespace
- In python, there are different types of namespaces:
 - ▶ built-in namespaces
 - ▶ global namespaces
 - ▶ local namespaces

- Contains the names of built-in functions and objects
- It is created while starting the python interpreter
- Exists as long as the interpreter runs
- Is destroyed when we close the interpreter
- It contains the names of built-in:
 - data types like integer or float
 - data structures like lists or dictionaries
 - exceptions and error names
 - functions like `print()` and `type()`

- Global namespaces are defined at the program or module level
- It contains the names of objects defined in a module or the main program
- A global namespace is created when the program starts
- Exists until the program is terminated by the python interpreter
- This namespace contains all our variables, data structures and functions we create when executing our script

- Defined for a class or a function
- The names defined within a class or function are local to it
- These variable names cannot be accessed outside the class or function in which they are defined
- The local namespace is created when the function or class starts executing
- Terminates when the function or class terminates

Multiple Returns

- We learned that we can use return to hand over the results of our function to the main script
- Until now we always had only one value we returned from our functions
- Python however allows us to hand back more than just one value by writing them after our return statement while comma separating them
- In the example below I hand back the results of an addition and subtraction:

```
def mathfunc(a,b):  
    sumup = a + b  
    subtrac = a - b  
    return sumup, subtrac
```

```
def mathfunc(a,b):  
    sumup = a + b  
    subtrac = a - b  
    return sumup, subtract
```

```
res1,res2 = mathfunc(5,6)  
results = mathfunc(5,6)
```

- We learned that we can assign the returned value of a function to a new variable
- This still holds true even if our function returns multiple values, however we need to declare the same amount of new variables as values we return from our function
- Alternatively we can declare exactly one new variable, which is then a tuple containing our returned values


```
def mathfunc(a,b):  
    sumup = a + b  
    subtrac = a - b  
    return sumup, subtract
```

```
res1,res2 = mathfunc(5,6)  
results = mathfunc(5,6)
```

- Please keep in mind that the order of the return statement determines which element / variable contains which value
- For our example res1 contains the value of sumup and res2 the value of subtract
- In results the first element is the value of sumup and the second element the value of subtract

- In exercise 44 you created three different functions, now combine them into a new function that returns all results at once
- Call this function at least three times with different arguments and print out the results
- Call the function at least once by specifying only 1 variable name on the left side of the „=„ sign

- Write a function that takes a list as parameter
- It should calculate the length, sum, minimum and maximum value of this list
- Return all four results to the main script
- Call the function at least four times with different lists
- Does it work with a list of strings as well?

Default Parameters

- Until now when we created our functions we used so called positional parameters, that means we always needed to hand over values to our function for them on the call
- We can however also define so called default parameters in our signatures by directly assigning a value to them
- These parameters can only be defined after our positional parameters

```
def sum_up(num1, num2=10):  
    result = num1 + num2  
    return result
```

- In the example above num1 is the positional parameter and num2 is a default parameter, that means if we don't hand over a value for it on the call of our function, the value of 10 is used instead

- So we could now create the following calls:

```
res1 = sum_up(5, 6)  
res2 = sum_up(5)
```

- Both work just fine, the first one returns 11 while the second one returns 15 (5+10)

- Write a script that contains a function that takes five numbers as parameters
- Three of them should be using a default value of 0, 10 and 100 respectively
- Add up all parameters
- Return the result to the main script
- Call the function at least four times
 - once with two arguments
 - once with three arguments
 - once with four arguments
 - once with five arguments

Keyword arguments

- We learned that the order of the arguments you hand over to your functions is essential and has to match the order of your parameters
- However there is a method to circumvent this order
- By using keyword arguments during the call of a function you can change the order of how you hand over values to your function
- If we think back to our function `sum_up` with it's two parameters `num1` and `num2`, there we saw that during a call as shown below `num1` would be assigned the value 5 and `num2` the value 6

```
res1 = sum_up(5, 6)
```

- Instead we can also write a call like this to tell Python which value should be assigned to which parameter:

```
res2 = sum_up(num2=5, num1=6)
```

- In this case num2 is assigned the value 5 and num1 is assigned the value 6 instead
- Using calls like this can especially useful to understand how call backs and the calculations inside your script work

- Use the function from Exercise 49 and add several calls to it using keyword arguments
- Assuming you named your parameters num1, num2, num3, num4 and num5 use the following orders for your calls:
 - ▶ num5, num4, num3, num2, num1
 - ▶ num4, num2, num5, num3, num1
 - ▶ num3, num4, num2, num1, num5
 - ▶ num1, num4, num2, num5, num3
 - ▶ num2, num5, num1, num3, num4

args of functions

- Allows you to pass a varying number of positional arguments
- Thus the function works with any number of arguments and you don't need to specify an exact amount of parameters
- Syntax:
 - `def function_name(*args):`
- Keep in mind that the name “args” is changeable the main thing is the “*”
- Keep also in mind that the *args object is a tuple and NOT a list

- Let's say you want to create a function that sums up all numbers you hand over to it
- Normally you would need to define the exact number of parameters in the signature
- With the help of *args you don't need to
- Instead the function now works with any number of arguments on call back
- The function simply creates a tuple out of the handed arguments and you can work with it

```
def my_sum(*args):  
    result = 0  
    for x in args:  
        result += x  
    return result  
  
res1 = my_sum(1, 2, 3)  
res2 = my_sum(5, 7, 9, 10, 5)
```

- Write a script that contains two functions
- The first one should add together varying amounts of numbers, to calculate the overall sum
 - ▶ However, this function should take at least two arguments (two positional parameters num1 & num2)
- The second function should merge (add together) any number of strings
- Call each function at least three times

Lambda?

- Lambda stems from the language lambda calculus developed by Alonzo Church in the 1930
- This language could encode any computations and is the basis for all functional languages
- Functional Languages work mainly with functions
- Python is not inherently a functional language, but it adopted some functional concepts early on
- Therefore in 1994 the lambda operator was added to Python



- Imagine a small function that just adds 1 to it's argument x:

```
def add1(x):  
    return x + 1
```

- This is a function like you already saw and that does its job just fine

- Afterwards we can call back both functions in the same way:

```
res1 = add1(2)           ➔           res1 = 3
```

- However we could achieve the same with the help of the lambda operator and just one line of code:

```
add1 = lambda x: x + 1
```

- This line does essentially exact the same as the function add1 we defined on the left side

- On the previous slide you saw how to use lambda to create a function that takes just one argument and returns a simple result
- When you created your first own functions however you used two parameters / arguments for your functions
- Of course lambda is able to achieve something like this as well:

```
add_up = lambda x,y : x+y
```

- When talking about lambda functions we need to define some terminology to make sure we are talking about the correct things
- Let's take the following lambda function:

```
add_up = lambda x, y : x+y
```

- In this example we have the the following definitions:
 - ▶ add_up is the function name
 - ▶ lambda is the keyword that tells Python that we define a function
 - ▶ x,y are the bound variables, that means they are arguments to our lambda function
 - ▶ x+y is the body of our lambda function, that means the calculations done by our function

- Take the functions from Exercise 44 and use lambda to make one-line functions that fulfill the same role
- Call each of the functions at least twice

Additional Keywords

- In Python there exist 3 additional keywords aside things like def, if or for you should know something about:
 - break
 - continue
 - pass
- Each of them has different uses which we will discuss on the following slides
- Try to use them only if you really need them

- This command completely stops your most inner loop and can thus only be used inside loops
- If you try to use it outside of loops you will produce an error, so be careful here
- You can use it if you are searching for something and don't need to run over the rest of the loop
- This is mainly used to reduce the running time of programs if you are only interested in certain parts of the calculation or if the calculation inside your loop reaches a point where it isn't necessary to go on anymore

- Continue can be used to skip parts of a loop and thus can also be used inside of loops and would produce errors otherwise
- All commands that are inside the same loop as a continue but come after this keyword are skipped and the loop goes on with the next iteration
- You can use this if you don't want your program to go over multiple if statements or other loops nested inside your loop as soon as a certain criterium is fulfilled
- Use it only if you can be sure that you don't need the remaining calculations in the current iteration

- Pass can be used as a placeholder for future code
- In python empty code is not allowed in function definitions, loops, if statements or class definitions
- You should use this only in combination with comments that tell what you want to do in this place later on
- I use it often when I'm constructing my programs
 - ▶ That means I create the loops, if statements and functions as empty parts and add the necessary code later on
 - ▶ Like this you can test out your code step by step, while having the overall structure ready

- Create a list with at least 15 entries you choose
- Iterate over it with a for loop and add up the elements
- Create one if statement in your loop that can't be true and add a pass
- Include a second if statement in your loop that breaks the loop if your sum gets bigger than a threshold (you chose)
- Include a third if statement in your loop that uses continue if the current entry of your list is in a specific range, e.g. 50 to 60, and prints out something in all other cases (use else)
- Print out the sum at the end

Definitions

- The **Signature** of a function defines the name of the function itself as well as the names and order of parameters it needs to fulfill its task
- Variables that are used by functions are called **Parameters**, their names are defined in the signature of the function
- Executing a function in a script by using the name of the function after it was defined is named **Calling** a function
- Values we hand over to a function during a call that should be used as parameters are named **Arguments**
- Handing back results from functions to the main script is called the **Return** of variables

- Variables that can be used through out the whole script after declaring them are called **Global Variables**
- Variables which are declared inside of functions are called **Local Variables** and are only usable inside of these functions
- Every variable, data structure or function we create has a **Name** which uniquely identifies it
- A **Namespace** is the combination of all names in a certain environment, for example a function
- Parameters that always need an argument during the call of a function are called **Positional Parameters**

- Parameters which don't need arguments to be handed over during a call are named **Default Parameters** and they always have to be defined after positional parameters in the signature of a function

1. https://en.wikibooks.org/wiki/Non-Programmer%27s_Tutorial_for_Python_3 (Last edited: 05.09.2022, Last visited: 02.01.2023)
2. <https://www.pythonforbeginners.com/basics/what-is-namespace-in-python> (Last edited: 18.08.2021, Last visited: 03.01.2023)
3. <https://techvidvan.com/tutorials/python-namespace-and-scope/> (Last visited: 03.01.2023)
4. <https://www.programiz.com/python-programming/function-argument> (Last visited: 03.01.2023)
5. <https://realpython.com/python-lambda/> (Last edited: 19.06.2019, Last visited: 14.04.2023)
6. <https://realpython.com/python-kwargs-and-args/> (Last updated: 04.09.2019, Last visited: 05.01.2023)